F4: SAT on the bit-level x-only Montgomery ladder, conflict-count scaling and clause reuse

Run F4-RUN-02 of study `sat_ladder_scaling_20261110`. Class: boundary diagnostic; no S, no rho row, no leaderboard change. **Status: run-02 complete; the replication prescribed by rule 2 is pending.** All numbers below are `[pcc:unverified]` (no pcc claim id exists for this run). No improvement over rho, or over anything, is claimed.

Purpose and mechanism

The study measures how the number of CDCL conflicts needed to recover a scalar k from $x(Q) = x([k]G)$ grows with the field size n, when the problem is a SAT instance of the bit-level projective x-only Montgomery ladder (the $\mathrm{nk} = \lceil \log_2 r \rceil$ bits of k are the only free inputs; each step is a differential add and a double, Tseitin-encoded with schoolbook multipliers and modular reduction by quotient variables). The frozen reference is exhaustive-like ($\alpha \approx 1$) against rho's 0.5. CaDiCaL 3.0.1, default options, one prime-order curve per n, 10 targets per n.

Arm FRESH: a new solver per target. Arm REUSE: one incremental solver per curve, target x_Q bits as assumptions, same targets in a seeded random order, learned clauses persist. Clauses learned on one target are advice for later ones, so amortisation falls under $S \cdot T^2 \gtrsim n$. Metric: conflicts; wall time descriptive only; censored solves (budget $4 \cdot 10^6$) are reported as censored, never imputed.

Frozen predictions and rules (PROTOCOL.md sections 3 and 4)

- **P1.** Exponent α of median FRESH conflicts in n ($\log_2$ conflicts $\approx \alpha n + c$, fitted on the uncensored n): $\alpha \in [0.85, 1.15]$; rho is 0.5. **P2.** REUSE/FRESH median conflict ratio at each $n \geq 0.5$. **P3.** All models decode to the true k. Prior that α upper CI < 0.6: 0.01.

1. **Instrument failure:** P3 fails, or fewer than three n are uncensored.
2. **Lead alive:** α upper 95 % CI < 0.6 with $n = 18$ or 20 uncensored, **or** REUSE ratio < 0.2 at two consecutive n. Then replicate with a second solver and 40 targets.
3. **Closed:** otherwise. Reading: CDCL on the ladder circuit is exhaustive-like and clause reuse does not amortise below the generic bound at these sizes.

Amendment before the run (section 6, summarised)

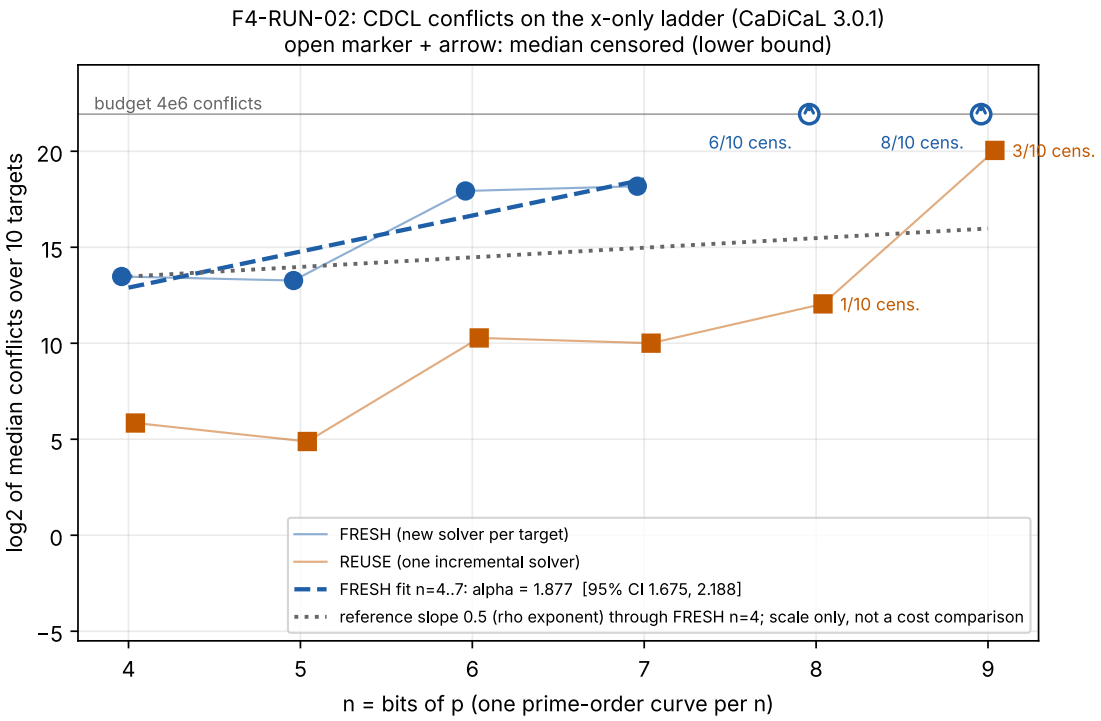
Predictions and rules were committed (`c8bbc80e1a`) before the tool existed. Development runs (not committed, not scored) showed about 5x more conflicts per bit, and at $n = 10, 12$ every solve censored at $2 \cdot 10^4$; the frozen grid $n \in \{10..20\}$ at 10^7 would have been censored throughout and tripped rule 1 by construction (a grid-design error). Amendment (grid and budget only): $n \in \{4..9\}$, 10 targets per n (not 20), budget $4 \cdot 10^6$, `--threads 2`. P1-P3 and rules 1-3 unchanged. x-only decoding accepts $k' \in \{k, r - k\}$. Expectation stated before the run: α_{FRESH} well above 1, P1 failing upward.

Result table `[pcc:unverified]`

Median conflicts over 10 targets (`results/run-02.stdout.txt`); cens = censored count of 10; ratio = REUSE median / FRESH median (n/a where the FRESH median is censored).

n	vars	clauses	FRESH med	cens	REUSE med	cens	ratio
4	6562	31326	11393.5	0	57.5	0	0.005
5	8651	41523	9856.0	0	29.5	0	0.003
6	17833	90762	250058.5	0	1240.5	0	0.005
7	23079	115102	296736.0	0	1027.0	0	0.003
8	36220	188187	cens	6	4240.0	1	n/a
9	43236	227543	cens	8	1080637.0	3	n/a

$\alpha_{\mathrm{FRESH}} = 1.877$, 95 % CI [1.675, 2.188] over $n = 4..7$; $\alpha_{\mathrm{REUSE}} = 1.787$, 95 % CI [0.365, 3.904] over $n = 4..7$. P1 false (upward, as stated beforehand), P2 false, P3 true: 102 of 120 solves found a model, all verified and decoded to $k' \in \{k, r - k\}$; 18 censored. `[pcc:unverified]`



Source: results/run.json (summary, fits). 10 targets per n and arm; budget 4e6; local shared host. [pcc:unverified]

Figure 1: log2 median conflicts vs n, FRESH and REUSE. Open markers with arrows: median censored (lower bound; censored counts annotated). Dashed: FRESH fit on $n = 4..7$ with its α and CI. Dotted: reference slope 0.5 through FRESH $n = 4$, for scale only, not a cost comparison. Run F4-RUN-02, 10 targets per n, budget $4 \cdot 10^6$, local shared host. Source: `results/run.json`; `figures/plot.py`.

Rule application `[pcc:unverified]`

Rule 1: no (P3 holds, four uncensored n). Rule 2: yes, by its clause-reuse clause (ratio < 0.2 at every n from 4 to 7); the alpha clause does not fire (upper CI 2.188). So "replicate with a second solver and 40 targets" applies: prescribed, **pending**, not carried out here.

REUSE arm by solve order `[pcc:unverified]`

Derived from `rows` of `results/run.json`; targets identical across arms. Conflicts in solve order 0..9 (c = censored at $4 \cdot 10^6$).

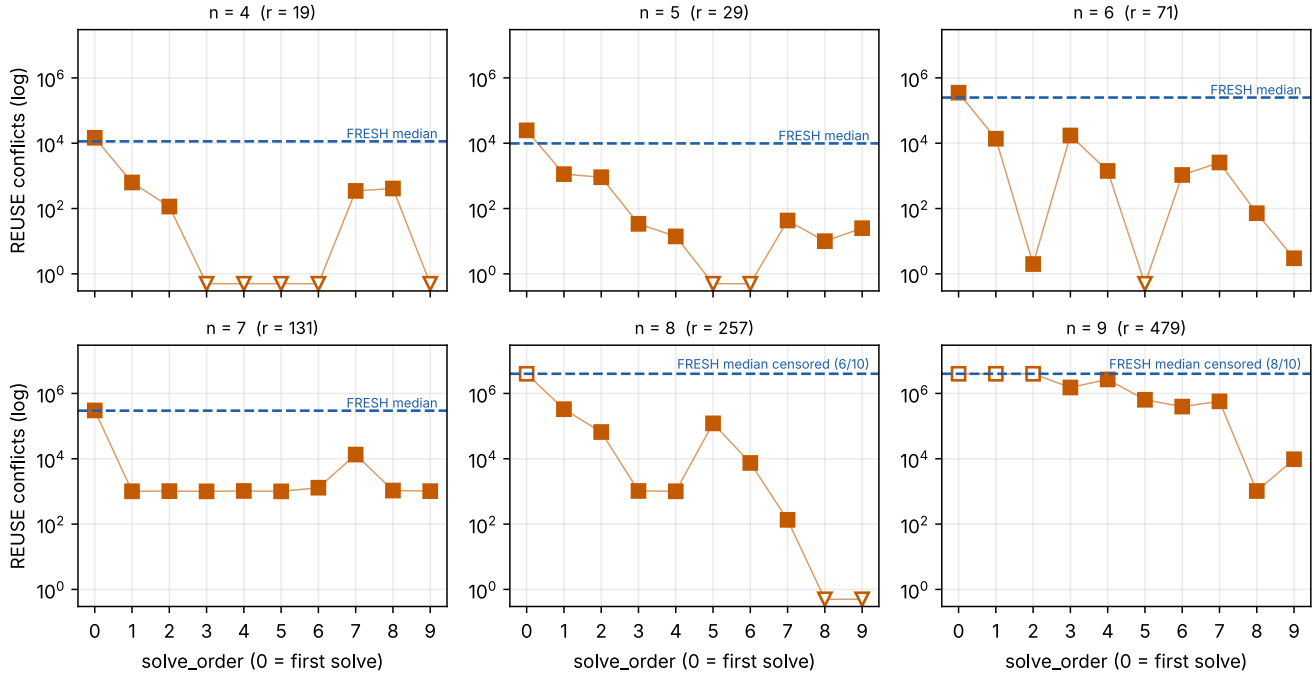
n	r	REUSE conflicts by solve_order 0, 1, ..., 9 (c = censored at 4e6)
4	19	14679, 628, 115, 0, 0, 0, 0, 346, 411, 0
5	29	24724, 1131, 909, 34, 14, 0, 0, 43, 10, 25
6	71	351983, 13732, 2, 17207, 1417, 0, 1064, 2570, 72, 3
7	131	301358, 1011, 1021, 1004, 1033, 1005, 1300, 13439, 1064, 1021
8	257	4000001 c, 329277, 65830, 1033, 1010, 121267, 7447, 135, 0, 0
9	479	4000000 c, 4000003 c, 4000000 c, 1520918, 2683356, 640356, 395495, 572444, 1030, 9661

First solve against later solves (warm median = median of the nine values of orders 1-9; computed here, not in the summary):

n	first solve	FRESH median	warm median (orders 1-9)	warm censored	distinct x_Q of 10 targets / (r-1)/2
4	14679	11393.5	0	0	7 / 9
5	24724	9856.0	25	0	7 / 14
6	351983	250058.5	1064	0	10 / 35
7	301358	296736.0	1021	0	9 / 65
8	4000001 c	cens	1033	0	10 / 128
9	4000000 c	cens	640356	2	10 / 239

The first solve costs about a FRESH solve (n = 4..7: 1.29, 2.51, 1.41, 1.02 times the FRESH median); later solves cost far less, with zeros at n = 4..6 and 8 and a flat band near 1000 at n = 7 (one 13,439); at n = 8 later conflicts fall along the order; at n = 9 the first three solves censor, orders 3 to 7 cost 3.95e5 to 2.68e6, orders 8 and 9 are 1030 and 9661.

F4-RUN-02: REUSE conflicts per solve, by solve order. Filled square: solved; open square: censored at 4e6; open triangle at the floor: 0 conflicts (plotted at 0.5)



Source: results/run.json (rows). 10 targets per n; FRESH median from summary. [pcc:unverified]

Figure 2: REUSE conflicts per solve vs solve order, per n (log scale; 0 conflicts drawn at a floor of 0.5, open triangles; open squares censored). Dashed: FRESH median (at the budget where censored). Run F4-RUN-02, 10 targets per n. Source: `results/run.json`; `figures/plot.py`.

Two readings, stated neutrally

- **Table exhaustion of a tiny group.** The group orders are 19, 29, 71, 131, 257, 479 and the 10 targets cover a sizeable share of the $\frac{r-1}{2}$ x-classes at small n (7 of 9; 7 of 14). Learned clauses could amount to a partial table of the group built during the first solve; later solves would be lookups, with warm cost growing with r.
- **Amortised inversion.** The clauses could encode reusable structure of the ladder circuit, so that a warm solve is cheaper than the first by a factor that does not simply track r.

The run does not separate them. **Discriminating quantity:** warm per-target cost against the group order r, over more targets and a second solver; the replication measures it. `[pcc:unverified]`

Limitations

Local host, 4 vCPU, shared CPU (concurrent cards; cargo lock waits in the receipt); the isolab-eks route was not used; wall time is descriptive only. 10 targets per n (20 frozen, amended); x_Q values not all distinct (7, 7, 10, 9, 10, 10 distinct), so the effective sample at $n = 4, 5$ is smaller. $n = 8, 9$ censored for FRESH (6 and 8 of 10), so the fit has four points; the REUSE CI is very wide. x-only decoding accepts $k' \in \{k, r - k\}$. Nothing measured at 18 or 20 bits. Single solver, single curve per n. No research-kb query; no novelty claim.

Evidence pointers

Card	`CARD-sat-ladder-scaling-20261110-F4-RUN-02`, sha256 `89d464edf4e769b2f51d2304deefabd8057662a13884946d444748df3639a604`
Receipt	`cards/receipts/F4-RUN-02.receipt.json` (completed, 13 commands, exit 0); assessment `F4-RUN-02.assessment.md`
Pinned head	`9465664dbe6ba5757b9cb011192215a00605e8d8`
Binary	sha256 `8e6cc0b7945ee2658ec2f92bb4a7a56ec7d2e79edb02316e8e2332792aef0ca0`
run.json	sha256 `a12f6c56bf42ba816a54149b13eb3cfd6d97751a68225f3331be2c7cd1425079` (120 rows)
FREEZE.sha256	PROTOCOL.md `156fe81023a340fa7431f870b3f60a58457fa236a57325100ec582f514c0bbbc`; tool/src/main.rs `6627a93246f7f4b0445a9a380d4f2e9d016ea001a0b7b671ac328f5e18c5a9b1`; tool/Cargo.toml `b94869ee38dcd3f7038ddd9b467e481d6ba51d821057765f391348985a53b0a8`; tool/Cargo.lock `b4685bb42c0da6592fa867809a194bdfee5c2c4b63c4b0d5d3f6674d8cb496b7` (all OK in the receipt)
Tests	7 passed, 0 failed, 1 ignored (`F4-RUN-02.test.stdout`)
Report build	typst Python wheel 0.15.2 (scratch venv); figures by matplotlib 3.11.2